

# Programozáselmélet - Párhuzamos programok

Készítette: Borsi Zsolt

Továbbra is szekvenciális programokkal foglalkozunk, de bevezetünk három új programkonstrukciót. Ezek definícióját úgy adjuk meg, hogy a velük képzett relációk megfeleljenek a program korábban kimondott definíciónak. Azaz, a három új programkonstrukció segítségével további programokat tudunk előállítani. Mivel célunk továbbra is az, hogy belássuk programok helyességét (tehát hogy igazoljuk hogy adott program megold egy adott feladatot), az új programkonstrukcióknak is megadjuk a levezetési szabályait.

## 1. Multiprogramozási modell

Három új programkonstrukció egyike a párhuzamos blokk, ami lehetővé teszi párhuzamosan végrehajtott programok reprezentálását. Egy párhuzamos program végrehajtása alatt a párhuzamos blokkot alkotó programok elemi utasításainak valamilyen ütemezés szerinti sorbarendezeit és egy processzoron (magon) való egymás utáni végrehajtását értjük. A multiprogramozási modellben az elemi programok végrehajtása (így az értékadás is) atomi módon történik, mint ahogy a logikai feltételek kiértékelése sem szakítható meg.

## 2. Új programkonstrukciók

- Atomi utasítás:  $[S]$   
Megszakítás nélkül hajtjuk végre.  $S$  nem tartalmazhat ciklust vagy várakoztató utasítást.
- Várakoztató utasítás: `await  $B$  then  $S$  ta`  
Amennyiben a  $\beta$  őrfeltétel igaz, az  $S$  program atomi utasításként a  $\beta$  kiértékelésével egyszerre hajtódik végre.  $S$  nem tartalmazhat ciklust vagy várakoztató utasítást. Ha  $\beta$  kiértékelhető de hamis, az utasítást tartalmazó folyamat blokkoltá válik; a végrehajtás nem tud továbblépni a várakoztató utasításon, amíg egy másik folyamat nem „segít” és változtatja meg az állapotot úgy hogy  $\beta$  teljesüljön.
- Párhuzamos blokk: `parbegin  $S_1 \parallel \dots \parallel S_n$  parend`  
Befejeződik, ha minden komponensprogramja terminál. Végrehajtása valamely ágának kiválasztását és az ott lévő komponens első utasításának, illetve a kapott maradék programnak egymás utáni végrehajtását jelenti.

**Definíció:** Legyen  $A$  tetszőleges állapotter,  $S$  program az  $A$  állapotter felett.

$\forall a \in A : [S](a) = S(a)$

**Definíció:** Legyen  $A$  közös állapottere az  $S$  programnak és a  $\beta$  logikai függvénynek (őrfeltételnek).

$$\forall a \in A : (\text{await } \beta \text{ then } S \text{ ta})(a) = \begin{cases} [S](a) & , \text{ ha } a \in \mathcal{D}_\beta \wedge \beta(a) \\ (\text{skip}; \text{await } \beta \text{ then } S \text{ ta})(a) & , \text{ ha } a \in \mathcal{D}_\beta \wedge \neg\beta(a) \\ \{< a, \text{fail} >\} & , \text{ ha } a \notin \mathcal{D}_\beta \end{cases}$$

Amikor az  $n$  komponensből álló párhuzamos blokkot végrehajtjuk, akkor egy adott állapotból indulva bármely  $S_i$  komponenssel ( $i \in [1..n]$ ) megkezdődhet a párhuzamos program végrehajtása. Amennyiben az  $S_i$  programot atomiként kell végrehajtani (mert vagy elemi program, vagy a  $[]$  atomi művelet programkonstrukcióval nem megszakíthatóvá lett téve), akkor annak végrehajtása után a **parbegin**  $S_1 \parallel \dots \parallel S_{i-1} \parallel S_{i+1} \parallel \dots \parallel S_n$  **parend** párhuzamos blokkot kell elvégezni.

Amennyiben  $S_i$  végrehajtása megszakítható, akkor  $S_i$  program felbontható egy  $u$  atomi utasításra és egy  $T$  programra. Vagyis  $S_i$  felfogható az  $u; T$  szekvenciaként, ahol  $u$  az  $S_i$  első (nem megszakítható) utasítása,  $T$  pedig az  $S_i$  maradék része. Ekkor a párhuzamos blokk egy végrehajtását kapjuk az  $u$  utasítás majd azt követően a **parbegin**  $S_1 \parallel \dots \parallel S_{i-1} \parallel T \parallel S_{i+1} \parallel \dots \parallel S_n$  **parend** párhuzamos blokkot elvégezve.

Mivel a párhuzamos blokk végrehajtását bármely komponens kiválasztva elkezdhetjük, a párhuzamos programok szükségképpen nem-determinisztikusak. Az összes végrehajtási sorozatot megkapjuk a párhuzamos blokk elvégzését az elsőként az  $S_i$  komponens aktiválásával kapott végrehajtási sorozatok összességéként.

**Definíció:** Legyen  $A$  közös alap-állapottere az  $S_1 \dots S_n$  programoknak.

$$\forall a \in A : (\text{parbegin } S_1 \parallel \dots \parallel S_i \parallel \dots \parallel S_n \text{ parend})(a) = \bigcup_{i=1}^n B_i(a)$$

ahol

$$B_i(a) = \begin{cases} (S_i; \text{parbegin } S_1 \parallel \dots \parallel S_{i-1} \parallel S_{i+1} \parallel \dots \parallel S_n \text{ parend})(a) & \text{ha } S_i \text{ nem megszakítható az } a \text{ állapotból indulva} \\ (u_i; \text{parbegin } S_1 \parallel \dots \parallel S_{i-1} \parallel T_i \parallel S_{i+1} \parallel \dots \parallel S_n \text{ parend})(a) & \text{ha } S_i(a) = (u_i; T_i)(a) \text{ ahol } u_i \text{ nem megszakítható az } a \text{ állapotból indulva} \end{cases}$$

Lásd a fejezet végén lévő kiegészítés részt, hogy mit értünk a különböző programszerkezetek első utasításaként egy adott  $a$  állapotból indulva.

### 3. Az új programkonstrukciók levezetési szabályai

**Tétel:** Atomi művelet levezetési szabálya

*Ha*

$Q \implies lf(S, R)$  akkor  $Q \implies lf([S], R)$

**Tétel:** Várakozó utasítás levezetési szabálya

*Ha*

1.  $Q \implies \beta \vee \neg\beta$

2.  $Q \wedge \beta \implies lf(S, R)$

akkor  $Q \implies lf(\text{await } \beta \text{ then } S \text{ ta}, R)$

**Tétel:** Párhuzamos blokk levezetési szabálya

Legyenek  $Q, Q_1, \dots, Q_n$  és  $R, R_1, \dots, R_n$  logikai függvények. *Ha*

1.  $Q \implies Q_1 \wedge \dots \wedge Q_n$  és

2.  $R_1 \wedge \dots \wedge R_n \implies R$  és

3.  $\forall i \in [1..n] : Q_i \implies lf(S_i, R_i)$  és

4. a  $Q_1 \implies lf(S_1, R_1), \dots, Q_n \implies lf(S_n, R_n)$  teljes helyességi formulák interferencia-mentesek és

5. a párhuzamos blokk holtpontmentes

akkor  $Q \implies lf(\text{parbegin } S_1 \parallel \dots \parallel S_n \text{ parend}, R)$

A párhuzamos blokk levezetési szabályának első három feltétele szemléletesen azt fejezi ki, hogy

- ha  $Q$  előfeltétel teljesül, akkor a párhuzamos blokk bármely  $S_i$  komponensprogramjának végrehajtása elkezdhető, mert teljesül a  $Q_i$  előfeltétele (ezt nevezhetjük „belépési feltételnek”).
- amikor a párhuzamos blokk terminál (az összes  $S_i$  komponens befejeződött, elérte utófeltételét) akkor jó helyen terminált: ott ahol  $R$  utófeltétel igaz (ezt nevezhetjük „kilépési feltételnek”).
- minden  $S_i$  komponens önmagában teljesen helyes a  $Q_i$  előfeltételével és  $R_i$  utófeltételével adott feladatra nézve.

## 4. Interferencia-mentesség

Ezzel a fogalommal azt akarjuk kifejezni, hogy ha valamit már beláttunk az  $S_i$  komponens esetén, az a bizonyítás nem veszti érvényét egy  $S_i$  komponenssel párhuzamosan végrehajtott  $S_j$  komponensben lévő  $u$  utasítás elvégzésével. Például, ne legyen az hogy miután már beláttuk hogy  $S_i$  egy ciklusának magjában a termináló függvény értéke csökken, a bizonyításunkat tönkreteszi  $u$  végrehajtása, azáltal hogy meg tudja növelni az adott ciklus termináló függvényének értékét. Különben érvényét vesztené az a bizonyításunk, melyben beláttuk hogy a ciklus terminál.

Mivel értéket megváltoztatni, és így egy logikai állítást hamissá tenni csak az értékadás tud, így  $u$  utasításként azon utasításokat kell figyelembe venni, amik vagy értékadások vagy atomi módon végrehajtott programként értékadást tartalmaznak. Nevezzük őket kritikus utasításnak!

**Jelölés:** Teljes helyességi formula Ha valamilyen  $S$  program és  $Q, R$  logikai függvények esetén teljesül hogy  $Q \implies lf(S, R)$ , akkor nevezzük az  $Q \implies lf(S, R)$  alakot az  $S$  egy teljes helyességi formulájának! Mivel az interferencia-mentesség igazolásához azt akarjuk megmutatni hogy egy bizonyítás nem veszti érvényét,  $S$  program helyett egy hozzá tartozó  $Q \implies lf(S, R)$  teljes helyességi formulát veszünk figyelembe (tehát azt amellyel bizonyítottuk hogy  $S$  program a  $Q$  feltételnek eleget tevő állapotokból indulva biztos hogy helyesen terminál, és a végállapotokban teljesül  $R$ ).

**Definíció:** A  $Q_1 \implies lf(S_1, R_1), \dots, Q_n \implies lf(S_n, R_n)$  teljes helyességi formulák interferencia-mentesek, ha bármely  $i$  és  $j$  esetén ( $i, j \in [1..n]$  és  $i \neq j$ ) az  $S_i$  komponens egyik kritikus utasítása sem interferál a  $Q_j \implies lf(S_j, R_j)$  teljes helyességi formulával.

**Definíció:** Azt mondjuk hogy az  $S_i$  komponens  $u$  kritikus utasítása (aminek előfeltételét jelölje  $pre_u$ ) nem interferál az  $Q_j \implies lf(S_j, R_j)$  teljes helyességi formulával, ha

- $pre_u \wedge R_j \implies lf(u, R_j)$   
Azaz,  $u$  végrehajtása nem rontja el  $S_j$  utófeltételét: ha  $R_j$  igaz volt a végrehajtás előtt akkor utána is igaz lesz.
- $pre_u \wedge pre_s \implies lf(u, pre_s)$  bármely  $S_j$ -beli  $s$  utasítás (aminek előfeltételét jelölje  $pre_s$ ) esetén  
Azaz  $u$  végrehajtása igaznak tartja meg  $S_j$  bármely  $s$  utasításának előfeltételét: ha  $s$  elvégezhető volt  $u$  végrehajtása előtt akkor utána is az lesz.
- $pre_u \wedge t = t_0 \implies lf(u, t \leq t_0), \forall t_0 \in \mathbb{Z}$  esetén ahol  $t$  egy  $S_j$ -beli ciklus termináló függvénye

Azaz  $u$  végrehajtása  $S_j$  bármely ciklusának  $t$  terminálófüggvényének értékét nem növeli meg.

## 5. Holtpont

Legyen  $A = (a:\mathbb{Z}, b:\mathbb{Z})$  és tekintsük az  $A$  alap-állapottér feletti következő programot:

```
a:=0; b:=0;
parbegin
  while IGAZ do
     $\alpha_0$ : a:=a+1;
     $\alpha_1$ : await  $b \neq 0$  then
       $\alpha_2$ : a:=a+3
    ta;
  od
||
  while IGAZ do
     $\beta_0$ : a:=2*a;
     $\beta_1$ : await  $a \neq 1$  then
       $\beta_2$ : b:=b+1
    ta;
  od
parend
```

Ez a program az állapotter bármely állapotából indulva, sorrendben a  $\beta_0$  majd  $\alpha_0$  címkével jelzett utasításokat végrehajtva, az  $\{a:1, b:0\}$  állapotba kerülve holtpontba jut.

**Definíció:** *Holtpont*

- Egy várakoztató utasítás egy adott állapotban blokkolt, ha az őrfeltétele hamis az állapotban.
- Egy szekvenciális program blokkolt, ha egy általa közvetlenül tartalmazott várakoztató utasítás blokkolt.
- Egy párhuzamos blokk holtpontban van, ha minden még be nem fejeződött komponense blokkolttá vált.

## 6. A holtpontmentesség egy elégséges feltétele

Elégséges feltételt szeretnénk arra adni, hogy a programunk garantáltan nem juthat holtpontra, nincs olyan végrehajtása amely során holtponthelyzet állna fent. Hangsúlyozzuk, hogy egy általános párhuzamos program alatt olyan programot értünk ami az eddig megengedett programkonstrukciókból épül fel (nagy valószínűséggel szekvencia) de tartalmazza az új programkonstrukciók valamelyikét is.

Tekintsünk egy ilyen programot. A holtpontmentesség szempontjából számunkra a várakoztató utasítások és a párhuzamos blokkok az érdekesek. Tegyük fel hogy az általános programunkban  $m$  olyan várakoztató utasítás szerepel ami nincs egy párhuzamos blokkon belül, a párhuzamos blokkok száma pedig legyen  $n$ .

A  $k$ -adik párhuzamos blokkot jelöljük így (tehát  $n_k$  komponense van a  $k$ -adik párhuzamos blokknak):

$T_k$ : **parbegin**  $S_1^k \parallel \dots \parallel S_{n_k}^k$  **parend**, ahol a komponensek akár újabb várakoztató utasításokat tartalmazhatnak.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $S:$<br>$\vdots$<br>$A_1: \text{await } \beta_1 \text{ then } S_1 \text{ ta};$<br>$\vdots$<br>$A_i: \text{await } \beta_i \text{ then } S_i \text{ ta};$<br>$\vdots$<br>$T_1: \text{parbegin } S_1^1 \parallel \dots \parallel S_{n_1}^1 \text{ parend}$<br>$\vdots$<br>$A_j: \text{await } \beta_j \text{ then } S_j \text{ ta};$<br>$\vdots$<br>$T_n: \text{parbegin } S_1^n \parallel \dots \parallel S_{n_n}^n \text{ parend}$<br>$\vdots$<br>$A_m: \text{await } \beta_m \text{ then } S_m \text{ ta};$<br>$\vdots$ |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Sorra vesszük a lehetséges holtpont helyzeteket:

- Az  $S$  szekvenciális folyamat blokkolt, ha valamely (nem párhuzamos blokkon belüli) várakoztató utasításánál várakozunk. Ezt úgy írjuk le hogy az  $A_j$  ( $j \in [1..m]$ ) várakoztató utasítást készek vagyunk elvégezni (a  $pre(A_j)$  előfeltétele teljesül) de a  $\beta_j$  őrfeltétel *hamis*.

- A szekvenciális  $S$  program valamely párhuzamos blokkja blokkolt. Tegyük fel hogy a  $T_k$  párhuzamos blokkról van szó. Azaz  $T_k$  minden  $S_i^k$  ( $i \in [1..n_k]$ ) komponensprogramja terminált vagy blokkolt, de legalább az egyik blokkolt.

Definiáljuk  $D(S)$ -et és  $D1(T_k)$ -et a következőképpen: (A  $D1$  alkalmazásakor tudjuk hogy az argumentuma egy párhuzamos blokk.)

•

$$D(S) = \left[ \bigvee_{j=1}^m (pre(A_j) \wedge \neg \beta_j) \right] \vee \left[ \bigvee_{k=1}^n D1(T_k) \right]$$

A formula azt fejezi ki hogy van benne (de nem párhuzamos blokkon belüli, hanem közvetlenül a „főprogramban”) blokkolt várakoztató utasítás, vagy valamely párhuzamos blokkja blokkolt.

•

$$D1(T_k) = \left[ \bigwedge_{j=1}^{n_k} (post(S_i^k) \vee D(S_i^k)) \right] \wedge \left[ \bigvee_{i=1}^{n_k} D(S_i^k) \right]$$

A  $T_k$  párhuzamos blokk blokkolt, ha minden  $S_i^k$  komponense blokkolt vagy véget ért, miközben van legalább egy komponense ami blokkolt.

**Tétel:** Amennyiben  $D(S)$  azonosan  $HAMIS$ , nem lehet az  $S$  programnak olyan végrehajtása hogy holtpont előfordulhatna;  $S$  holtpontmentes.

## 7. Kiegészítés: $S$ program felbontása első utasításra és az azt követő maradék programra

Itt megadjuk hogy írható fel egy  $a$  kezdőállapotot tekintve az  $S$  program (ahol  $S$  nem elemi program és nem is atomiként végrehajtható) az  $u; T$  szekvenciaként, ahol  $u$  az  $S$  nem megszakítható első utasítása,  $T$  pedig a maradék program.

- Ha  $S = \text{await } \beta \text{ then } S_0 \text{ ta}$  és  $a \in \mathcal{D}_\beta \wedge \neg \beta(a)$ , akkor  $u$  a **SKIP** program míg  $T$  az  $\text{await } \beta \text{ then } S_0 \text{ ta}$  várakoztató utasítás.
- Ha  $S = (S_1; S_2)$  és  $S_1$  végrehajtása nem megszakítható  $a$ -ból indulva, akkor  $u$  az  $S_1$  és  $T$  az  $S_2$ .
- Ha  $S = (S_1; S_2)$  és  $S_1$  végrehajtása megszakítható  $a$ -ból indulva, akkor  $u$  az  $S_1$  első (atomiként végrehajtható) utasítása, míg  $T$  a  $(T_1; S_2)$  szekvencia (ahol  $T_1$  az  $S_1$  maradék része).

- Ha  $S = \text{if } \pi_1 \rightarrow S_1 \square \dots \square \pi_n \rightarrow S_n \text{ fi}$  és van olyan  $\pi_i$  feltétel ami nem értelmezett  $a$  állapotban vagy mindegyik feltétel hamis  $a$ -ban ( $\exists i \in [1..n] : a \notin \mathcal{D}_{\pi_i} \vee \forall i \in [1..n] : a \in \mathcal{D}_{\pi_i} \wedge \neg \pi_i(a)$ ), akkor  $u$  az **ABORT** program és  $T$  szintén az **ABORT**.
- Ha  $S = \text{if } \pi_1 \rightarrow S_1 \square \dots \square \pi_n \rightarrow S_n \text{ fi}$  és az  $a$  állapotban mindegyik feltétel értelmezett és legalább egy közülük teljesül ( $\forall i \in [1..n] : a \in \mathcal{D}_{\pi_i} \wedge \exists i \in [1..n] : \pi_i(a)$ ), akkor  $u$  a **SKIP** program míg  $T$  az  $S_i$  (feltéve hogy azt a végrehajtást nézzük ami az  $a$  állapotból indulva az  $i$ -edik ág kiválasztásával történik).
- Ha  $S = \text{while } \pi \text{ do } S_0 \text{ od}$  és az  $a$  állapotban a ciklusfeltétel nem értelmezett ( $a \notin \mathcal{D}_\pi$ ), akkor  $u$  és  $T$  is az **ABORT** program.
- Ha  $S = \text{while } \pi \text{ do } S_0 \text{ od}$  és  $a$  állapotban a ciklusfeltétel hamis ( $a \in \mathcal{D}_\pi \wedge \neg \pi(a)$ ), akkor  $u$  és  $T$  is az **ABORT** program.
- Ha  $S = \text{while } \pi \text{ do } S_0 \text{ od}$  és a ciklusfeltétel teljesül az  $a$  állapotban ( $a \in \mathcal{D}_\pi \wedge \pi(a)$ ), akkor  $u$  a **SKIP** program és  $T$  az  $(S_0; \text{while } \pi \text{ do } S_0 \text{ od})$  szekvencia.
- Ha  $S = \text{parbegin } S_1 \parallel \dots \parallel S_i \parallel \dots \parallel S_n \text{ parend}$  és az  $a$  állapotból indulva nem megszakíthatóan végrehajtható  $S_i$  elvégzésével kezdjük meg a párhuzamos blokk végrehajtását, akkor  $u$  az  $S_i$  és  $T$  a  $\text{parbegin } S_1 \parallel \dots \parallel S_{i-1} \parallel S_{i+1} \parallel \dots \parallel S_n \text{ parend}$  párhuzamos blokk.
- Ha  $S = \text{parbegin } S_1 \parallel \dots \parallel S_i \parallel \dots \parallel S_n \text{ parend}$  és az  $a$  állapotból indulva a megszakítható  $S_i$  elvégzésével kezdjük meg a párhuzamos blokk végrehajtását, akkor  $u$  az  $S_i$  első (atomi módon végrehajtható) utasítása, és  $T$  az  $\text{parbegin } S_1 \parallel \dots \parallel S_{i-1} \parallel T_i \parallel S_{i+1} \parallel \dots \parallel S_n \text{ parend}$  párhuzamos blokk, ahol  $T_i$  az  $S_i$  maradék programja.

# Programozáselmélet - Levezetési szabályok

Készítette: Borsi Zsolt

**Tétel:** A szekvencia levezetési szabálya

Legyen  $A$  közös alap-állapotterű  $S_1$  és  $S_2$  programok szekvenciája  $S = (S_1; S_2)$ . Legyenek  $Q, Q'$  és  $R$  logikai függvények  $A$ -n. Ha

$$1. Q \implies lf(S_1, Q') \text{ és}$$

$$2. Q' \implies lf(S_2, R)$$

akkor  $Q \implies lf(S, R)$ .

**Tétel:** Az elágazás levezetési szabálya

Legyen  $IF = (\pi_1:S_1, \dots, \pi_n:S_n)$  a közös  $A$  alap-állapotterű  $S_i$  programokból képzett,  $A$  feletti  $\pi_i$  logikai függvényekkel meghatározott elágazás. Legyenek továbbá  $Q$  és  $R$  logikai függvények  $A$ -n. Ha

$$1. Q \implies \bigwedge_{i=1}^n (\pi_i \vee \neg \pi_i) \text{ és}$$

$$2. Q \implies \bigvee_{i=1}^n \pi_i \text{ és}$$

$$3. \forall i \in [1..n] : Q \wedge \pi_i \implies lf(S_i, R)$$

akkor  $Q \implies lf(IF, R)$ .

**Tétel:** A ciklus levezetési szabálya

Legyen  $DO = (\pi, S_0)$  az  $A$  alap-állapotterű feletti  $S_0$  programból és a  $\pi \in A \rightarrow \mathbb{L}$  feltétellel képzett ciklus. Továbbá legyenek  $P, Q$  és  $R$  logikai függvények  $A$ -n és  $t: A \rightarrow \mathbb{Z}$  függvény adottak. Ha

$$1. Q \implies P \text{ és}$$

$$2. P \wedge \neg \pi \implies R \text{ és}$$

$$3. P \implies \pi \vee \neg \pi \text{ és}$$

$$4. P \wedge \pi \implies t > 0 \text{ és}$$

$$5. P \wedge \pi \implies lf(S_0, P)$$

$$6. P \wedge \pi \wedge t = t_0 \implies lf(S_0, t < t_0) \text{ bármely } t_0 \text{ egész számra}$$

akkor  $Q \implies lf(DO, R)$ .

A levezetési szabályban szereplő  $P$  állítást a ciklus invariáns tulajdonságának, a  $t$  függvényt terminálófüggvénynek nevezzük.

Az utolsó két pont összevonható:

$$5 - 6. \quad P \wedge \pi \wedge t = t_0 \implies lf(S_0, P \wedge t < t_0) \text{ bármely } t_0 \text{ egész számra}$$

# Programozáselmélet - Programkonstrukciók

Készítette: Borsi Zsolt

Ebben a fejezetben azzal foglalkozunk, hogy lehet meglévő programokból új programokat készíteni. Háromféle konstrukciót engedünk meg: szekvencia, elágazás és ciklus. Ezek definícióit úgy adjuk meg, hogy a velük képzett relációk illeszkedjenek a korábban bevezetett program fogalmához. A programkonstrukciókat struktogrammal ábrázoljuk.

## 1. Szekvencia

Szekvencia esetében két programot egymás után végzünk el. Amennyiben az első program végrehajtása adott állapotból indulva nem véges vagy hibásan terminál, a második program nem tudja folytatni a végrehajtást a végpontból; az első program által generált végrehajtás egy lehetséges végrehajtása lesz a szekvenciának is.

Nevezzük csatlakozási pontnak azt az állapotot egy végrehajtási sorozatban, ahol az egyik program terminál és a másik program ugyaninnen indul el. Nem szeretnénk ha például az  $x := x + 1$  és  $x := x + 2$  programok szekvenciája (ahol  $x$  egész típusú) az állapottér  $\{x:5\}$  eleméhez az  $\langle \{x:5\}, \{x:6\}, \{x:6\}, \{x:8\} \rangle$  sorozatot rendelné. Ezért az egymás után véges sokszor ismétlődő csatlakozási pontokból egyet hagyjunk el.

**Jelölés:** Véges hosszú  $\alpha$  sorozat utolsó elemét jelölje  $\tau(\alpha)$ . Tehát ha  $\alpha$  véges,  $\tau(\alpha) = \alpha_{|\alpha|}$ .

**Jelölés:** Legyen  $A$  tetszőleges állapottér.  $\alpha \in \bar{A}^*$  és  $\beta \in (\bar{A} \cup \{fail\})^{**}$  úgy hogy  $\alpha$  és  $\beta$  nem üres sorozatok továbbá  $\tau(\alpha) = \beta_1$ . Ekkor  $\alpha \otimes \beta$  jelölje az  $\alpha$  és  $\beta$  sorozatok összefűzésében  $\beta$  első elemének elhagyásával kapott sorozatot.

Általánosítsuk a jelölést  $n \in \mathbb{N}^+$  darab vagy akár végtelen sok sorozat esetére. A sorozatok összefűzése (konkatenációja) után az egymás után véges sokszor ismétlődő csatlakozási pontokból egyet hagyjunk el.

**Példa:** Legyen  $A = \{1, 2, 3, 4\}$ . Ekkor

$$\otimes_4(\langle 1, 2, 3, 1 \rangle, \langle 1, 2, 3, 1 \rangle, \langle 1, 2, 3, 1 \rangle, \langle 1, 2, 3, 1 \rangle) = \langle 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1 \rangle$$

$$\otimes_4(\langle 1 \rangle, \langle 1 \rangle, \langle 1 \rangle, \langle 1, 2, 3, 1 \rangle) = \langle 1, 1, 1, 2, 3, 1 \rangle$$

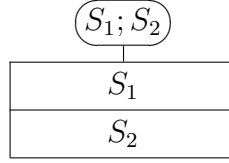
$$\otimes_\infty(\langle 1 \rangle, \langle 1 \rangle, \langle 1 \rangle, \langle 1 \rangle, \dots) = \langle 1, 1, \dots \rangle$$

$$\otimes_\infty(\langle 1, 2, 3, 1 \rangle, \langle 1, 2, 3, 1 \rangle, \langle 1, 4 \rangle, \langle 4 \rangle, \langle 4 \rangle, \langle 4 \rangle, \dots) = \langle 1, 2, 3, 1, 2, 3, 1, 4, 4, 4, \dots \rangle$$

**Definíció:** Legyen  $A$  közös alap-állapottére az  $S_1$  és  $S_2$  programoknak. Az  $(S_1; S_2)$  relációt az  $S_1$  és  $S_2$  programok szekvenciájának nevezzük, ha

$$\begin{aligned} (S_1; S_2)(a) = & \{\alpha \in \bar{A}^\infty \mid \alpha \in S_1(a)\} \cup \\ & \{\alpha \in (\bar{A} \cup \{fail\})^* \mid \alpha \in S_1(a) \wedge \alpha_{|\alpha|} = fail\} \cup \\ & \{\gamma \in (\bar{A} \cup \{fail\})^{**} \mid \gamma = \alpha \otimes \beta \wedge \alpha \in S_1(a) \wedge |\alpha| < \infty \wedge \alpha_{|\alpha|} \neq fail \wedge \beta \in S_2(\alpha_{|\alpha|})\} \end{aligned}$$

A szekvencia struktogramja:



**Tétel:** Legyen  $A$  közös alap-állapottere az  $S_1$  és  $S_2$  programoknak. Az  $(S_1; S_2)$  szekvencia program.

**Tétel:** Legyen  $A$  közös alap-állapottere az  $S_1$  és  $S_2$  programoknak és jelölje  $S$  az  $(S_1; S_2)$  szekvenciát. Ekkor

$$p(S) = p(S_2) \odot p(S_1)$$

## 2. Elágazás

**Definíció:** Legyen  $A$  közös alap-állapottere az  $S_1, \dots, S_n$  programoknak. Legyenek továbbá  $\pi_1, \dots, \pi_n \in A \rightarrow \mathbb{L}$  logikai függvények. Ekkor az  $IF \subseteq A \times (\bar{A} \cup \{fail\})^{**}$  relációt az  $S_i$  programokból képzett  $\pi_i$  feltételek által meghatározott elágazásnak nevezzük és  $(\pi_1:S_1, \dots, \pi_n:S_n)$ -nel jelöljük, ha

$$\forall a \in A : IF(a) = \omega_0(a) \cup \bigcup_{i=1}^n \omega_i(a)$$

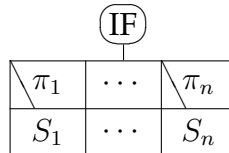
ahol  $\forall i \in [1..n] :$

$$\omega_i(a) = \begin{cases} S_i(a), & \text{ha } a \in \mathcal{D}_{\pi_i} \wedge \pi_i(a) \\ \emptyset, & \text{ha } a \in \mathcal{D}_{\pi_i} \wedge \neg \pi_i(a) \\ \{ \langle a, fail \rangle \}, & \text{ha } a \notin \mathcal{D}_{\pi_i} \end{cases}$$

és

$$\omega_0(a) = \begin{cases} \{ \langle a, fail \rangle \}, & \text{ha } \forall i \in [1..n] : (a \in \mathcal{D}_{\pi_i} \wedge \neg \pi_i(a)) \\ \emptyset, & \text{különben} \end{cases}$$

Az elágazás struktogramja:



Az elágazást szokás még a következő módon is leírni:

```

if
   $\pi_1 \rightarrow S_1 \square$ 
  ...
   $\pi_{n-1} \rightarrow S_{n-1} \square$ 
   $\pi_n \rightarrow S_n$ 
fi

```

**Tétel:** Legyen  $A$  közös alap-állapottere az  $S_1, \dots, S_n$  programoknak. Legyenek továbbá  $\pi_1, \dots, \pi_n \in A \rightarrow \mathbb{L}$  logikai függvények. Az  $IF = (\pi_1:S_1, \dots, \pi_n:S_n)$  elágazás program.

**Tétel:** Legyen  $A$  közös alap-állapottere az  $S_1, \dots, S_n$  programoknak. Legyenek továbbá  $\pi_1, \dots, \pi_n \in A \rightarrow \mathbb{L}$  logikai függvények.  $IF = (\pi_1:S_1, \dots, \pi_n:S_n)$ . Ekkor

$$\mathcal{D}_{p(IF)} = \{a \in A \mid a \in \bigcap_{i=1}^n \mathcal{D}_{\pi_i} \wedge a \in \bigcup_{i=1}^n [\pi_i] \wedge \forall i \in [1..n] : a \in [\pi_i] \implies a \in \mathcal{D}_{p(S_i)}\}$$

és

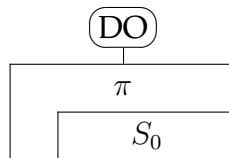
$$\forall a \in \mathcal{D}_{p(IF)} : p(IF)(a) = \bigcup_{i=1}^n p(S_i)|_{[\pi_i]}$$

### 3. Ciklus

**Definíció:** Legyen  $\pi \in A \rightarrow \mathbb{L}$  feltétel és  $S_0 \subseteq A \times (\bar{A} \cup \{fail\})^{**}$  program. A  $DO \subseteq A \times (\bar{A} \cup \{fail\})^{**}$  relációt az  $S_0$  programból  $\pi$  feltétellel képzett ciklusnak nevezzük és  $(\pi, S_0)$ -al jelöljük, ha  $\forall a \in A$ :

$$DO(a) = \begin{cases} (S_0; DO)(a) & \text{ha } a \in \mathcal{D}_{\pi} \wedge \pi(a) \\ \{< a >\} & \text{ha } a \in \mathcal{D}_{\pi} \wedge \neg\pi(a) \\ \{< a, fail >\} & \text{ha } a \notin \mathcal{D}_{\pi} \end{cases}$$

A ciklus struktogramja:



A szakirodalomban a következő mód is elterjedt a ciklus leírására:

```

while  $\pi$  do
   $S_0$ 
od

```

A szekvenciához és az elágazáshoz hasonló módon is definiálhatjuk a ciklust.

**Definíció:**  $\forall a \in A$ :

$$DO(a) = \begin{cases} \{ \langle a, fail \rangle \}, & \text{ha } a \notin \mathcal{D}_\pi \\ \{ \langle a \rangle \}, & \text{ha } a \in \mathcal{D}_\pi \wedge \neg \pi(a) \\ \{ \alpha \in (\bar{A} \cup \{fail\})^{**} \mid \exists \alpha^1, \dots, \alpha^n \in (\bar{A} \cup \{fail\})^{**}: \alpha = \otimes_n(\alpha^1, \dots, \alpha^n) \wedge \\ \alpha^1 \in S_0(a) \wedge \forall i \in [1..n-1]: (\alpha^i \in \bar{A}^* \wedge \tau(\alpha^i) \in [\pi] \wedge \alpha^{i+1} \in S_0(\tau(\alpha^i))) \wedge \\ ((\alpha^n \in \bar{A}^\infty \vee (\alpha^n \in (\bar{A} \cup \{fail\})^* \wedge \tau(\alpha^n) = fail)) \vee \\ (\alpha^n \in \bar{A}^* \wedge \tau(\alpha^n) \in \mathcal{D}_\pi \wedge \tau(\alpha^n) \notin [\pi])) \} \\ \cup \\ \{ \alpha \in \bar{A}^\infty \mid \exists \alpha^1, \alpha^2, \dots \in \bar{A}^*: \alpha = \otimes_\infty(\alpha^1, \alpha^2, \dots) \wedge \alpha^1 \in S_0(a) \wedge \forall i \in \mathbb{N}^+: \\ (\alpha^i \in \bar{A}^* \wedge \tau(\alpha^i) \in [\pi] \wedge \alpha^{i+1} \in S_0(\tau(\alpha^i))) \} \\ \cup \\ \{ \alpha \in (\bar{A} \cup \{fail\})^* \mid \exists \alpha^1, \dots, \alpha^n \in (\bar{A} \cup \{fail\})^{**}: \alpha = \otimes_n(\alpha^1, \dots, \alpha^n) \wedge \\ \alpha^1 \in S_0(a) \wedge \forall i \in [1..n-2]: (\alpha^i \in \bar{A}^* \wedge \tau(\alpha^i) \in [\pi] \wedge \alpha^{i+1} \in S_0(\tau(\alpha^i))) \\ \wedge (\alpha^{n-1} \in \bar{A}^* \wedge \tau(\alpha^{n-1}) \notin \mathcal{D}_\pi \wedge \alpha^n = \langle \tau(\alpha^{n-1}), fail \rangle) \}, & \text{ha } a \in \mathcal{D}_\pi \wedge \pi(a) \end{cases}$$

Első ránézésre a definíció kissé bonyolultnak tűnik. Amennyiben sorra vesszük hogy az állapot-tér egy  $a$  pontjához milyen sorozatokat rendelhet a ciklus, már nem is fogjuk bonyolultnak találni a definíciót!

- Ha  $a$  állapotban a ciklusfeltétel nem kiértékelhető, akkor a ciklus hibásan terminál.
- Ha  $a$  állapotban a ciklusfeltétel kiértékelhető de nem teljesül  $a$ -ra, akkor a ciklus semmit nem csinál, végrehajtása befejeződik az  $a$  állapotban.
- A ciklusmagot véges sokszor elvégezzük egymás után, úgy hogy a ciklusmag utolsó végrehajtásához tartozó sorozat
  - végtelen; vagy
  - véges és a  $fail$  állapotban végződik; vagy
  - véges és utolsó elemében kiértékelhető a  $\pi$  feltétel, de az *hamis*.
- A ciklusmagot végtelen sokszor elvégezzük egymás után, mert egy végrehajtás után mindig olyan állapotba jutunk ahol a ciklusfeltétel teljesül.
- Az utolsó lehetőség az, hogy a ciklusmagot azért véges sokszor (de legalább egyszer) végezzük el egymás után, mert utoljára egy olyan állapotba jutunk ahol a ciklusfeltétel nem kiértékelhető.

# Programozásméletek - A specifikáció tétele

Készítette: Borsi Zsolt

## 1. Specifikáció tétele

**Definíció:** Azt mondjuk hogy a  $B$  halmaz az  $F \subseteq A \times A$  feladat egy paramétertere, ha léteznek olyan  $F_1 \subseteq A \times B$  és  $F_2 \subseteq B \times A$  relációk, melyekre  $F = F_2 \circ F_1$ .

**Megjegyzés:** Bármely  $F \subseteq A \times A$  feladatnak létezik paramétertere. Hiszen legyen  $B = A$ , és válasszuk az  $F_1 \subseteq A \times B$  és  $F_2 \subseteq B \times A$  relációkat úgy, hogy  $F_1 = id$  (azaz a minden  $A$ -beli elemhez önmagát rendelő reláció) és  $F_2 = F$ . Ekkor nyilvánvalóan teljesül hogy  $F \circ id = F$ .

**Definíció:** Legyenek  $A$  és  $B$  tetszőleges nemüres halmazok és  $R \subseteq A \times B$  tetszőleges reláció. Az  $R$  reláció inverze:

$$R^{(-1)} ::= \{(b, a) \in B \times A \mid (a, b) \in R\}$$

azaz olyan  $B$ -ről  $A$ -ra képező reláció, ami pontosan akkor tartalmaz egy  $(b, a) \in B \times A$  párt, ha  $(a, b) \in R$ .

**Tétel:** Legyen  $F \subseteq A \times A$  feladat,  $B$  az  $F$  egy paramétertere (azaz léteznek olyan  $F_1 \subseteq A \times B$  és  $F_2 \subseteq B \times A$  relációk úgy hogy  $F = F_2 \circ F_1$ ). Legyen  $b \in B$  tetszőleges paraméter, amihez definiáljuk a  $Q_b: A \rightarrow \mathbb{L}$  és  $R_b: A \rightarrow \mathbb{L}$  logikai függvényeket az igazsághalmazaik megadásával:

$$[Q_b] ::= F_1^{(-1)}(b)$$

$$[R_b] ::= F_2(b)$$

Ekkor ha  $\forall b \in B : Q_b \implies lf(S, R_b)$  akkor az  $S$  program megoldja az  $F$  feladatot.

$[Q_b] = \{a \in A \mid (a, b) \in F_1\}$ , azaz  $Q_b$  igazsághalmazában olyan  $a \in A$  állapotok vannak, melyekhez az  $F_1$  reláció hozzárendeli a  $b \in B$  paramétert.

$[R_b] = \{a \in A \mid (b, a) \in F_2\}$ , azaz  $R_b$  igazsághalmazában olyan  $a \in A$  állapotok vannak, melyeket az  $F_2$  reláció a  $b \in B$  paraméterhez rendel.

## 2. Feladat specifikációja

Tekintsük azt a feladatot, amikor egy adott pozitív egész egy osztóját kell megadnunk. A feladat állapottere  $A = (n:\mathbb{N}^+, d:\mathbb{N}^+)$ . Ezt a feladatot le tudjuk formálisan írni, mint olyan  $(u, v) \in A \times A$  párok halmaza ahol  $u$  és  $v$  állapotok  $n$  változó szerinti értékei megegyeznek és

$v$  célállapot  $d$  változóhoz tartozó értéke osztója az  $u$  kiindulási állapot  $n$  változóhoz tartozó értéknek:

$$\{(u, v) \in A \times A \mid n(u) = n(v) \wedge d(v) \mid n(u)\}$$

Felhasználva a specifikáció tételének jelöléseit, írjuk fel más módon - de formálisan - a feladat specifikációját. Azt vesszük észre, hogy minden  $a \in A$  állapothoz melyekre  $n(a)$  megegyezik, a feladat ugyanazt rendeli; nem függ a kiindulási állapot  $d$  változó szerinti értékétől. Írjuk fel az  $F$  feladatot a  $F_1$  és  $F_2$  relációk kompozíciójaként, úgy hogy azokhoz az állapotokhoz melyeknek  $F$  szerinti képe azonos,  $F_1$  ugyanazt a paramétert rendelje. Mivel ezek az állapotok megegyeznek az  $n$  változóhoz tartozó értékükben, célszerű hozzájuk ugyanezt a (címkézett) értéket rendelni,  $F_1$  reláció szerint. Azaz, a feladat egy paramétertere legyen a pozitív egész számok halmaza, ahol az értékre az  $n'$  változó segítségével (egy komponens lévén, nem lenne szükség változóra, de általános esetben kell) hivatkozhatunk:  $B = (n':\mathbb{N}^+)$ .

Azt, hogy  $F_1$  pontosan akkor rendel egy  $a \in A$  állapothoz egy  $b \in B$  értéket, a specifikáció tételében szereplő  $Q_b$  logikai függvénnyel adhatjuk meg. Legyen  $b \in B$  tetszőleges, ekkor  $\forall a \in A : Q_b(a) = (n(a) = n'(b))$ .

Természetesen úgy kapjuk meg  $F$  feladatot az  $F_1$  és  $F_2$  relációk kompozíciójaként, ha  $F_2$  a  $b \in B$  paraméterhez olyan állapottérbeli  $a$  pontot rendel, melynek  $d$  változóhoz tartozó értéke osztója a kiindulási állapot  $n$  változóhoz tartozó értékének. Épp ezért tetszőleges  $b \in B$  esetén legyen  $R_b$  olyan logikai függvény, melyre

$$\forall a \in A : R_b(a) = (n(a) = n'(b) \wedge d(a) \mid n(a)).$$

Vegyük észre hogy a  $n(a) = n'(b)$  kikötésre szükségünk van, anélkül csak annyit mondanánk hogy a célállapotban a  $d$  változóhoz tartozó érték osztója kell legyen az  $n$  változóhoz tartozó aktuális értéknek, nem fogalmaznánk meg szorosabb kapcsolatot a kiindulási állapot és a célállapot között. A feladat specifikációja tehát

$$A = (n:\mathbb{N}^+, d:\mathbb{N}^+)$$

$$B = (n':\mathbb{N}^+)$$

$$\forall b \in B : Q_b(a) = (n(a) = n'(b)) \text{ (ahol } a \in A \text{ tetszőleges állapot)}$$

$$\forall b \in B : R_b(a) = (n(a) = n'(b) \wedge d(a) \mid n(a)) \text{ (ahol } a \in A \text{ tetszőleges állapot)}$$

A továbbiakban a feladatnak ezen leírását (tehát ami tartalmazza a feladat állapotterét és a feladat egy paraméterterének megadását; minden  $b \in B$  paraméterre a hozzá tartozó  $Q_b$  és  $R_b$  logikai függvények definícióját) a feladat specifikációjának nevezzük. Mivel  $d$  az  $A$  állapottérre  $\mathbb{N}$ -re képező függvény (azaz argumentumába egy  $a \in A$  elemet írhatunk), hasonlóan  $Q_b$  egy  $b \in B$  paraméterhez definiált,  $a \in A$  állapotokhoz logikai értéket rendelő függvény; az egyértelmű jelöléseket elhagyva a következőket kapjuk:

$$A = (n:\mathbb{N}^+, d:\mathbb{N}^+)$$

$$B = (n':\mathbb{N}^+)$$

$$Q = (n = n')$$

$$R = (Q \wedge d \mid n)$$

# Programozáselmélet - A leggyengébb előfeltétel

Készítette: Borsi Zsolt

## 1. Nevezetes logikai függvények

**Definíció:** Legyen  $A$  tetszőleges halmaz.  $HAMIS$  jelöli azt a logikai függvényt, melyre

$$\forall a \in A: HAMIS(a) = \{hamis\}$$

**Definíció:** Legyen  $A$  tetszőleges halmaz.  $IGAZ$  jelöli azt a logikai függvényt, melyre

$$\forall a \in A: IGAZ(a) = \{igaz\}$$

Azaz a  $HAMIS$  logikai függvény egy adott  $A$  halmaz minden eleméhez a *hamis*, az  $IGAZ$  az *igaz* értéket rendeli.

**Jelölés** (Igazsághalmaz): Legyen  $R \in A \rightarrow \mathbb{L}$  logikai függvény. Ekkor  $\lceil R \rceil$  jelöli az olyan állapottérbeli pontok halmazát ahol  $R$  igaz. Azaz

$$\lceil R \rceil = \{a \in A \mid R(a) = \{igaz\}\}$$

Az  $\lceil R \rceil$  halmazt az  $R$  logikai függvény *igazsághalmazának* nevezzük.

Ne felejtsük el hogy  $R \in A \rightarrow \mathbb{L}$  nem feltétlenül értelmezett az  $A$  minden pontjában. Amennyiben  $R: A \rightarrow \mathbb{L}$ , akkor már viszont igaz hogy egy  $a \in A$  pontban ha  $R$  nem igaz, akkor hamis.

## 2. A „következik” reláció

**Definíció:** Legyenek  $Q, R \in A \rightarrow \mathbb{L}$  tetszőleges logikai függvények. Amennyiben  $\lceil Q \rceil \subseteq \lceil R \rceil$  teljesül, akkor azt mondjuk hogy  $Q$  maga után vonja  $R$ -t (vagy másképp:  $Q$ -ból következik  $R$ ) és a következőképpen jelöljük:  $Q \implies R$ .

Vegyük észre, hogy ha  $Q \implies R$ , akkor ez azt jelenti, hogy minden olyan  $a \in A$  pontra amire  $Q$  igaz, arra igaz  $R$  is.

**Példa:** Legyen  $A = \{1, 2, 3, 4\}$  és  $Q, R \in A \rightarrow \mathbb{L}$  logikai függvények úgy hogy  $\lceil Q \rceil = \{1, 3, 4\}$  és  $\lceil R \rceil = \{1, 3\}$ . Ebben az esetben  $Q \implies R$  nem teljesül (mert a 4-re igaz  $Q$  de  $R$  nem), de  $R \implies Q$  igen.

**Példa:** Legyen  $A = (a:\mathbb{N}, h:\mathbb{N})$  és  $Q, R \in A \rightarrow \mathbb{L}$  logikai függvények úgy hogy  $Q = (a = 10)$  és  $R = (h = a^3)$ . Ugyan van olyan  $A$ -beli pont (az  $A$  halmaz most speciálisan egy állapottér, tehát elemei állapotok) amihez  $Q$  és  $R$  is igazat rendel, méghozzá az  $\{a:10, h:1000\}$  állapot, de az nem igaz hogy  $Q \implies R$ , hiszen például  $\{a:10, h:82\} \in \lceil Q \rceil$ , de az  $\{a:10, h:82\}$  elemhez  $R$  hamisat rendel.

### 3. Leggyengébb előfeltétel

**Definíció:** Legyen  $S \subseteq A \times (\bar{A} \cup \{\text{fail}\})^{**}$  program,  $R \in A \rightarrow \mathbb{L}$  logikai függvény. Ekkor az  $S$  program  $R$  utófeltételhez tartozó leggyengébb előfeltétele az az  $lf(S, R): A \rightarrow \mathbb{L}$  függvény, amelyre

$$\lceil lf(S, R) \rceil = \{a \in A \mid a \in D_{p(S)} \wedge p(S)(a) \subseteq \lceil R \rceil\}$$

A leggyengébb előfeltétel tehát pontosan azokban a pontokban igaz, ahonnan kiindulva az  $S$  program biztosan hibátlanul terminál, és az összes lehetséges végállapotban igaz  $R$ .

**Tétel** (Az  $lf$  tulajdonságai): Legyen  $S \subseteq A \times (\bar{A} \cup \{\text{fail}\})^{**}$  program,  $Q, R \in A \rightarrow \mathbb{L}$  logikai függvények. Ekkor

1.  $lf(S, HAMIS) = HAMIS$
2. ha  $Q \implies R$  akkor  $lf(S, Q) \implies lf(S, R)$
3.  $lf(S, Q) \wedge lf(S, R) = lf(S, Q \wedge R)$
4.  $lf(S, Q) \vee lf(S, R) \implies lf(S, Q \vee R)$

**Példa:** Legyen  $A = (x:\mathbb{N})$ .  $R: A \rightarrow \mathbb{L}$  logikai függvény adott,  $R = (x < 10)$ . Számoljuk ki az  $x := x - 5$  értékadásnak az  $R$  utófeltételhez tartozó leggyengébb előfeltételét.

Először megvizsgáljuk hogyan viselkedik az  $x := x - 5$  program az állapottér néhány pontjában: az  $\{x:8\}$  ponthoz az  $\langle \{x:8\}, \{x:3\} \rangle$  sorozatot, míg az  $\{x:2\}$  állapothoz az  $\langle \{x:2\}, fail \rangle$  sorozatot rendeli. A program programfüggvénye olyan  $\{x:a_1\}$  állapotokban van értelmezve, ahol  $a_1 \geq 5$ , innen indulva a program garantáltan olyan pontban terminál ahol  $x$  értéke  $a_1 - 5$ . Egyéb állapotból indulva a program a  $fail$  állapotban terminál.

Felhasználva a leggyengébb előfeltétel definícióját, és az  $x := x - 5$  értékadást  $S$ -sel jelölve, felírhatjuk:

$$\begin{aligned} \lceil lf(S, R) \rceil &= \{a \in A \mid a \in D_{p(S)} \wedge p(S)(a) \subseteq \lceil R \rceil\} = \\ &= \{a \in A \mid x(a) \geq 5 \wedge \{x(a) - 5\} \subseteq \lceil R \rceil\} = \\ &= \{a \in A \mid x(a) \geq 5 \wedge x(a) - 5 \in \lceil R \rceil\} = \\ &= \{a \in A \mid x(a) \geq 5 \wedge x(a) - 5 < 10\} \end{aligned}$$

Azaz azt kaptuk, hogy az  $lf(S, R)$  pontosan akkor igaz ha  $(5 \leq x < 15)$ . Ne felejtsük el hogy az  $A$  állapottéren az egyetlen változónk neve  $x$  és most számoltuk ki azon állapotok halmazát ahol a leggyengébb előfeltétel igaz.

A leggyengébb előfeltétel fogalma nagyon fontos, ugyanakkor nagyon egyszerű. Vegyük észre hogy az előbbi példában azt számoltuk ki, hogy az  $x$  értéke 15-nél kisebb kell legyen, hogy az  $x := x - 5$  értékadást végrehajtva olyan pontban termináljunk ahol  $x$  értéke 10-nél kisebb. Továbbá az  $x$  értéke legalább 5 kell legyen, hogy az  $x := x - 5$  értékadás hibátlanul működjön (ne felejtsük: az  $x$  típusa természetes szám).

Természetesen igaz az is hogy  $x \in [8..12] \implies lf(x := x - 5, x < 10)$ , azaz hogy ha az  $x$ -hez tartozó érték a  $[8..12]$  halmazból van, akkor az  $x := x - 5$  biztos hogy hibátlanul terminál, még hozzá olyan állapotban ahol  $x < 10$  teljesül. Mindez azért van, mert az  $x \in [8..12]$  feltétel szigorúbb mint az a leggyengébb előfeltétel amit előbb kiszámoltunk.

Általánosan: ha valamely  $P$  logikai függvényre teljesül hogy  $P \implies lf(S, R)$  (azaz  $P$  szigorúbb mint az  $lf(S, R)$  feltétel) akkor a  $P$  tulajdonságú pontokból indulva az  $S$  program biztos hogy helyesen terminál és a végpontokban igaz  $R$ . A leggyengébb előfeltételt ezért hívják „leggyengébb előfeltételnek”.

# Programozáselmélet - A megoldás fogalma

Készítette: Borsi Zsolt

## 1. Programfüggvény

**Definíció:** A  $p(S) \subseteq A \times A$  reláció az  $S \subseteq A \times (\bar{A} \cup \{fail\})^{**}$  program *programfüggvénye*, ha

1.  $\mathcal{D}_{p(S)} = \{a \in A \mid S(a) \subseteq \bar{A}^*\}$
2.  $\forall a \in \mathcal{D}_{p(S)}: p(S)(a) = \{b \in A \mid \exists \alpha \in S(a): b = \alpha_{|\alpha|}\}$

## 2. Megoldás

**Definíció:** Azt mondjuk hogy az  $S$  program megoldja az  $F$  feladatot (más szavakkal: az  $S$  program teljesen helyes az  $F$  feladatra nézve), ha

1.  $\mathcal{D}_F \subseteq \mathcal{D}_{p(S)}$
2.  $\forall a \in \mathcal{D}_F: p(S)(a) \subseteq F(a)$

## 3. Parciális helyesség

**Definíció** (Gyenge programfüggvény): A  $\tilde{p}(S) \subseteq A \times (A \cup \{fail\})$  reláció az  $S \subseteq A \times (\bar{A} \cup \{fail\})^{**}$  program *gyenge programfüggvénye*, ha

1.  $\mathcal{D}_{\tilde{p}(S)} = \{a \in A \mid S(a) \cap (\bar{A} \cup \{fail\})^* \neq \emptyset\}$
2.  $\forall a \in \mathcal{D}_{\tilde{p}(S)}: \tilde{p}(S)(a) = \{b \in A \cup \{fail\} \mid \exists \alpha \in S(a) \cap (\bar{A} \cup \{fail\})^*: b = \alpha_{|\alpha|}\}$

**Definíció** (Parciális helyesség): Azt mondjuk hogy az  $S$  program parciálisan helyes az  $F$  feladatra nézve, ha

1.  $\forall a \in \mathcal{D}_F: \tilde{p}(S)(a) \subseteq F(a)$

# Programozáselmélet - Programozási alapfogalmak

Készítette: Borsi Zsolt

## 1. Állapottér

A feladat adatokról szól, a program is adatokkal dolgozik. Egy adat típusérték-halmaza az adat lehetséges értékeiből áll.

**Definíció** (Állapot): Legyenek  $A_1, \dots, A_n$  (ahol  $n \in \mathbb{N}^+$ ) típusérték-halmazok és  $v_1, \dots, v_n$  a halmazokat azonosító egyedi címkék (változók). Az ezekből képzett, címkézett értékeknek egy  $\{v_1:a_1, \dots, v_n:a_n\}$  halmazát (ahol  $\forall i \in [1..n] : a_i \in A_i$ ) *állapotnak* nevezzük.

Egy-komponensű állapottér esetén  $\{v_1:a_1\}$  helyett írhatunk  $a_1$ -et is.

**Definíció** (Állapottér): Legyenek  $A_1, \dots, A_n$  (ahol  $n \in \mathbb{N}^+$ ) típusérték-halmazok és  $v_1, \dots, v_n$  a halmazokat azonosító egyedi címkék (változók). Az ezekből képzett összes lehetséges  $\{v_1:a_1, \dots, v_n:a_n\}$  állapot (ahol  $\forall i \in [1..n] : a_i \in A_i$ ) halmazát *állapottérnek* nevezzük és  $(v_1:A_1, \dots, v_n:A_n)$ -nel jelöljük.

$$(v_1:A_1, \dots, v_n:A_n) ::= \{ \{v_1:a_1, \dots, v_n:a_n\} \mid \forall i \in [1..n] : a_i \in A_i \}$$

**Definíció** (Változó): Az  $A = (v_1:A_1, \dots, v_n:A_n)$  állapottér címkeire (*változók*) úgy tekintünk mint  $v_i : A \rightarrow A_i$  függvényekre, ahol  $v_i(a) = a_i$  egy  $a = \{v_1:a_1, \dots, v_n:a_n\}$  állapot esetén.

**Definíció** (Altér): Legyenek  $A = (v_1:A_1, \dots, v_n:A_n)$  és  $B = (u_1:B_1, \dots, u_m:B_m)$  állapottérek ( $n, m \in \mathbb{N}^+$  és  $m \leq n$ ). Azt mondjuk, hogy az  $A$  állapottérnek *altere* a  $B$  állapottér ( $B \leq A$ ), ha van olyan  $\varphi : [1..m] \rightarrow [1..n]$  injekció, amelyre  $\forall i \in [1..m] : B_i = A_{\varphi(i)}$ .

## 2. Feladat

**Definíció** (Feladat): Legyen  $A$  tetszőleges állapottér. Feladatnak nevezünk egy  $F \subseteq A \times A$  relációt.

A feladat fenti definíciója természetes módon adódik abból, hogy a feladatot egy leképezésnek tekintjük az állapottéren, és az állapottér minden pontjára megmondjuk, hova kell belőle eljutni, ha egyáltalán el kell jutni belőle valahova.

## 3. Sorozatok

**Jelölés:** Ha  $H$  tetszőleges halmaz, akkor jelölje  $H^{**}$  az olyan (akár véges, vagy akár végtelen) sorozatok halmazát, mely sorozatok elemei mind a  $H$  halmazból valók. A  $H$ -beli

véges sorozatok halmazát  $H^*$ -gal, a végtelen sorozatok halmazát  $H^\infty$ -nel jelöljük. Tehát  $H^{**} = H^* \cup H^\infty$ . Egy  $\alpha \in H^*$  sorozat hosszát jelölje  $|\alpha|$ , végtelen sorozat esetén legyen  $|\alpha| = \infty$ .

## 4. Program

**Definíció** (Program): Legyen  $A$  az úgynevezett alap-állapottér ( $fail \notin A$ ). Jelölje  $\bar{A}$  azon véges komponensű állapotterek unióját, melyeknek altere az  $A$  alap-állapottér:  $\bar{A} = \bigcup_{A \leq B} B$ .

Az  $A$  feletti programnak hívjuk az  $S \subseteq A \times (\bar{A} \cup \{fail\})^{**}$  relációt, ha

1.  $\mathcal{D}_S = A$
2.  $\forall a \in A: \forall \alpha \in S(a) : |\alpha| \geq 1$  és  $\alpha_1 = a$
3.  $\forall \alpha \in \mathcal{R}_S : (\forall i \in \mathbb{N}^+ : i < |\alpha| \rightarrow \alpha_i \neq fail)$
4.  $\forall \alpha \in \mathcal{R}_S : (|\alpha| < \infty \rightarrow \alpha_{|\alpha|} \in A \cup \{fail\})$

## 5. Elemi programok

**Definíció** (Elemi program): Legyen  $A$  tetszőleges állapotter. Az  $S \subseteq A \times (\bar{A} \cup \{fail\})^{**}$  programot *elemi programnak* nevezzük, ha

$$\forall a \in A: S(a) \subseteq \{ \langle a \rangle, \langle a, fail \rangle, \langle a, a, a, \dots \rangle, \langle a, b \rangle \mid b \in A \}$$

A definíció szerint minden programhoz található vele ekvivalens elemi program, csak a sorozatok közbülső elemeit el kell hagyni, így lényegében (egy adott szinten) minden program elemi. Az elemi programok közül kiválasztunk néhány speciális tulajdonsággal rendelkezőt, és a továbbiakban velük foglalkozunk.

**Definíció:** Legyen  $A$  tetszőleges állapotter.  $SKIP$  jelöli azt a programot, melyre

$$\forall a \in A: SKIP(a) = \{ \langle a \rangle \}$$

A  $SKIP$  az állapotter minden  $a$  állapotához egyetlen sorozatot, az  $\langle a \rangle$  sorozatot rendeli. Így  $a$ -ból indulva a  $SKIP$  garantáltan hogy  $a$ -ba jut, és csak oda.

**Definíció:** Legyen  $A$  tetszőleges állapotter.  $ABORT$  jelöli azt a programot, melyre

$$\forall a \in A: ABORT(a) = \{ \langle a, fail \rangle \}$$

Az  $ABORT$  az állapotter minden  $a$  állapotához egyetlen sorozatot, az  $\langle a, fail \rangle$  sorozatot rendeli. Így  $a$ -ból indulva az  $ABORT$  programnak nincs más végrehajtása, mint a  $fail$  állapotban végződő végrehajtás.

A harmadik speciális elemi program az értékadás, amivel az állapotter egyes komponenseinek (változóinak) értéke megváltoztatható.

# Programozáselmélet - Szükséges matematikai alapok

Készítette: Borsi Zsolt

## 1. Halmazok

Matematikában gyakran használt halmazok jelölései:

|                |                                                       |
|----------------|-------------------------------------------------------|
| $\mathbb{N}$   | – a természetes számok halmaza, a nullát is beleértve |
| $\mathbb{N}^+$ | – a pozitív egész számok halmaza                      |
| $\mathbb{Z}$   | – az egész számok halmaza                             |
| $\mathbb{L}$   | – a logikai értékek kételemű halmaza                  |
| $\emptyset$    | – az üreshalmaz                                       |

A halmazokat gyakran vagy az elemeik felsorolásával (például a logikai értékek halmaza: )

$$\mathbb{L} ::= \{igaz, hamis\}$$

vagy egy tulajdonság megfogalmazásával (például a 100-nál nem nagyobb 5-tel osztható egészek halmaza: )

$$\{x \in \mathbb{Z} \mid x \leq 100 \wedge 5|x\}$$

adjuk meg.

**Definíció** (Intervallum): Az  $[a..b] ::= \{x \in \mathbb{Z} \mid a \leq x \wedge x \leq b\}$  halmazt (ahol  $a$  és  $b$  egész számok) *intervallumnak* nevezzük. Ami üres, ha  $a > b$ .

**Jelölés:** Egy  $H$  halmaz számosságát  $|H|$  jelöli. Azt, hogy egy  $H$  halmaz véges, így is írhatjuk:  $|H| < \infty$ .

## 2. Reláció

**Definíció:** Legyenek  $A$  és  $B$  tetszőleges nemüres halmazok. Ekkor az  $A \times B$  halmaz az  $A$  és  $B$  Descartes szorzata, és

$$A \times B ::= \{(a, b) \mid a \in A \wedge b \in B\}$$

$A \times B$  elemei tehát olyan rendezett párok, ahol a pár első komponense  $A$ -ból, a második  $B$ -ből van.

**Definíció** (Reláció): Legyenek  $A$  és  $B$  tetszőleges nemüres halmazok. Ekkor az  $A \times B$  halmaz minden  $R$  részhalmazát (tehát az üreshalmazt is) *relációnak* nevezzük.

Ha  $(x, y) \in R$ , akkor azt mondjuk hogy az  $R$  reláció  $x$ -hez hozzárendeli  $y$ -t.

**Definíció:** Legyenek  $A$  és  $B$  tetszőleges nemüres halmazok és  $R \subseteq A \times B$  tetszőleges reláció. Az  $R$  reláció értelmezési tartománya:

$$\mathcal{D}_R ::= \{ a \in A \mid \exists b \in B : (a, b) \in R \}$$

a reláció értékkészlete:

$$\mathcal{R}_R ::= \{ b \in B \mid \exists a \in A : (a, b) \in R \}$$

a reláció értéke egy  $a$  helyen, vagy másképpen az  $a$  pont  $R$  reláció szerinti képe:

$$R(a) ::= \{ b \in B \mid (a, b) \in R \}$$

Az  $R \subseteq A \times B$  relációt felfoghatjuk egy leképezésnek, megfeleltetésnek is az  $A$  és a  $B$  halmaz elemei között.

### 3. Függvény

Speciális reláció a függvény.

**Definíció** (Függvény): Legyenek  $A$  és  $B$  tetszőleges nemüres halmazok és  $R \subseteq A \times B$  tetszőleges reláció. Azt mondjuk hogy az  $R$  reláció determinisztikus, ha

$$\forall a \in A : |R(a)| \leq 1$$

A determinisztikus relációkat másképpen függvényeknek hívjuk. Az  $R \subseteq A \times B$  függvénynek, mint speciális relációnak külön jelölést vezetünk be:  $R \in A \rightarrow B$ .

**Jelölés:** Ha az  $f \in A \rightarrow B$  függvényre még az is teljesül, hogy értelmezési tartománya megegyezik az  $A$  halmazzal, vagyis ha

$$\forall a \in A : |f(a)| = 1$$

akkor a következő jelölést alkalmazzuk:  $f: A \rightarrow B$ .

**Megjegyzés:** Az olyan  $f \in A \rightarrow B$  függvényeket melyek nem rendelnek minden  $A$ -beli elemhez egy  $B$ -beli elemet (vagyis nincsenek mindenhol értelmezve az  $A$  felett), parciális függvényeknek nevezzük.

**Megjegyzés:** Legyen  $f: A \rightarrow B$  függvény (tehát tudjuk hogy az  $f$  reláció az  $A$  halmaz minden eleméhez pontosan egy  $B$ -belit rendel) és  $a \in A$ . Legyen továbbá  $f(a) = \{b\}$  ahol  $b \in B$  az  $a$ -hoz rendelt egyetlen elem. Ebben az esetben az  $f(a)$  képet sokszor nem a  $\{b\}$  egyelemű képhalmazként hanem egyszerűen csak mint  $b$  írjuk.